Final Project

Transformer-Based Text Classification

Natural Language Processing

AIG230NAA

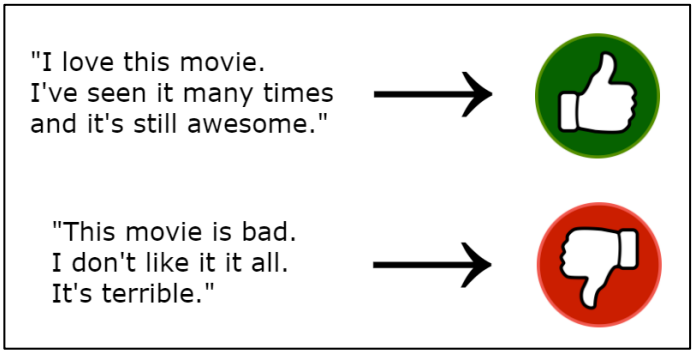
SENECA'S ACADEMIC INTEGRITY POLICY
As a Seneca student, you must conduct yourself in an honest and trustworthy manner in all aspects of your academic career. A dishonest attempt to obtain an academic advantage is considered an offense and will not be tolerated by the Polytechnic.

In this project, you will design and build a complete text classification system using a Transformer-based model. The objective is to develop a model capable of reading text and assigning it to the correct category, such as sentiment, topic, emotion, or news category. This project is intended to simulate a real-world NLP pipeline, requiring you to move beyond simply training a model and instead build a complete end-to-end machine learning solution.

It is recommended to create a virtual environment using virtualenv specifically for this project. The project can be completed in groups of up to 2 individuals.

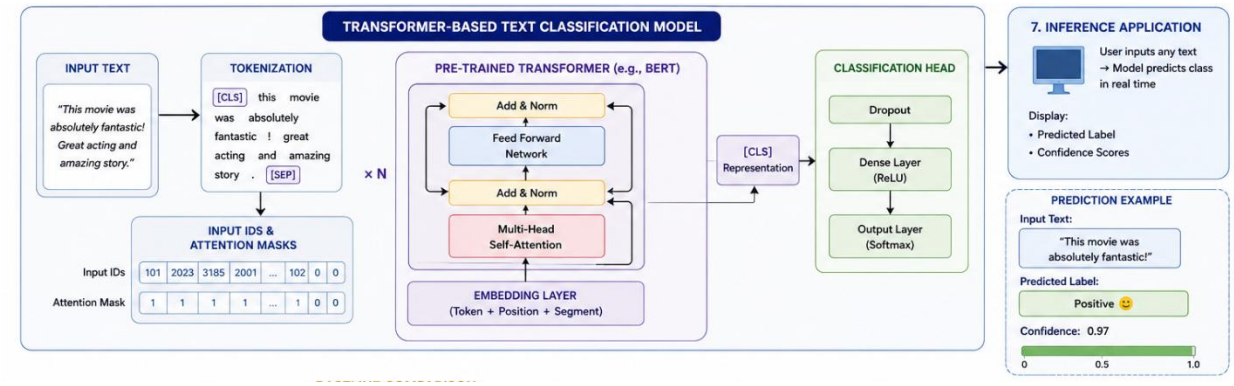
You are expected to complete every stage of the project, beginning with data collection and exploration, followed by preprocessing, model development, training, evaluation, error analysis, and finally the development of an interactive inference application. Your final submission should demonstrate not only the performance of your model, but also your understanding of the decisions made throughout the development process.

You may choose any publicly available text classification dataset, such as IMDb Movie Reviews, AG News, Yelp Reviews, Emotion Dataset, Fake News Detection Dataset, or Hate Speech Detection Dataset.



Your first task is to carefully explore the dataset before building the model. Provide a brief analysis describing the size of the dataset, number of classes, class distribution, average text length, vocabulary characteristics, and any imbalance or patterns that may affect classification performance. Supporting your observations with appropriate visualizations is highly encouraged.

Next, prepare the data for training by implementing a complete preprocessing pipeline. This includes cleaning the text where necessary, tokenizing the input, converting the text into the format required by the Transformer model, creating attention masks, padding or truncating sequences, encoding labels, and splitting the dataset into training, validation, and testing sets. Every preprocessing decision should be clearly justified.



You will then build a Transformer-based text classification model using TensorFlow or PyTorch. (You may want to fine-tune a pre-trained Transformer model such as BERT, DistilBERT, or RoBERTa. However, the training pipeline must be implemented by you rather than relying completely on high-level automated training utilities.) Your work should show that you understand how Transformer models process text and how the classification layer is used to make predictions.

Once the model has been prepared, train it using an appropriate optimization strategy and carefully monitor the learning process. You should present and discuss the training and validation curves, explain your choice of hyperparameters, and describe any techniques used to improve model performance, such as learning rate scheduling, dropout, checkpointing, or early stopping.

After training, evaluate your classification system using appropriate metrics such as accuracy, precision, recall, F1-score, and confusion matrix.

An important component of this project is error analysis. Rather than reporting only numerical results, examine at least twenty examples where the model makes incorrect predictions. Identify common failure cases such as ambiguous wording, sarcasm, mixed sentiment, long texts, rare vocabulary, class imbalance, or label noise. Discuss possible reasons for these errors and suggest ways the model could be improved.

Finally, you are required to develop a complete inference application that allows a user to interact with your trained model. The application should load the saved model, accept any new text entered by the user, predict the class without retraining the model, and display the predicted label in real time. The application should also show the confidence score or class probabilities when possible.

The final submission should include your complete source code, trained model weights, a well-organized report or README documenting your methodology and experimental results, and a demonstration of your inference application. It is highly recommended to record a short video of the final result at inference time. Your project will be evaluated not only on model performance, but also on the quality of your implementation, analysis, and ability to build a complete, production-style NLP system.

Deliverables

At the end of the project, you are expected to submit a complete package that demonstrates every stage of your work, from model development to deployment. The following items are required:

1. Source Code: Submit all source code used throughout the project.

2. Trained Model: Include the final trained model (weights/checkpoints) used during inference.

3. Project Report: Include a comprehensive README that explains how to install dependencies, set up the environment, train the model (if desired), load the trained model, and run the inference application.

4. Demonstration Video

Evaluation Rubric

Criteria	Weight	Poor	Fair	Good	Very Good	Excellent
Data Collection & Exploration	10%	Incomplete or poor dataset selection and analysis.	Basic exploration.	Good analysis.	Thorough analysis with visualizations.	Comprehensive and insightful analysis.
Data Preprocessing	10%	Incomplete or incorrect.	Basic preprocessing.	Mostly correct.	Well-designed pipeline.	Excellent, efficient preprocessing.
Transformer Model Development	25%	Incomplete or incorrect model.	Partially correct implementation.	Correct with minor issues.	Well-implemented architecture.	Excellent implementation and design.
Training Pipeline	15%	Incomplete or incorrect training.	Basic training pipeline.	Functional with appropriate settings.	Well-designed and monitored.	Robust, optimized training pipeline.
Evaluation & Baseline Comparison	15%	Incomplete evaluation.	Basic evaluation.	Appropriate metrics and comparison.	Strong evaluation and discussion.	Comprehensive and insightful evaluation.
Error Analysis	10%	Little or no analysis.	Basic analysis.	Reasonable insights.	Thorough analysis.	Deep and insightful analysis.
Inference Application	10%	Incomplete or non-functional.	Basic functionality.	Functional application.	User-friendly application.	Professional-quality application.
Code Quality	5%	Poor organization.	Basic organization.	Clean and readable.	Well-structured and documented.	Excellent, modular, professional code.

Resources to help:

- PEP8 – Style Guide for Python Code
- Machine Learning Project Structure
- The illustrated Transformer